Bank-Grade Confluence RAG on Bedrock AgentCore

An engineering blueprint for migrating the IFEX Confluence RAG system to AWS while maintaining Java-first development workflows, strict VPC boundaries, and zero-hallucination guardrails.

The Strategy: Java-First Portability on AWS AgentCore

The Stable Portability Layer (What you keep)

- **Graph config** (YAML/JSON)
- **Chunk store schema** (Postgres)
- **Vector index schema** (OpenSearch/Elastic)
- **Citation JSON contract** (Source cards)
- **Your agent logic remains in Java** (Spring Boot + Spring AI Advisors/LangGraph4j).

Your agent logic remains in Java (Spring Boot + Spring AI Advisors/LangGraph4j).

The AWS Hosting Layer (What changes)

Runtime moves to **AgentCore** (framework-agnostic container hosting).

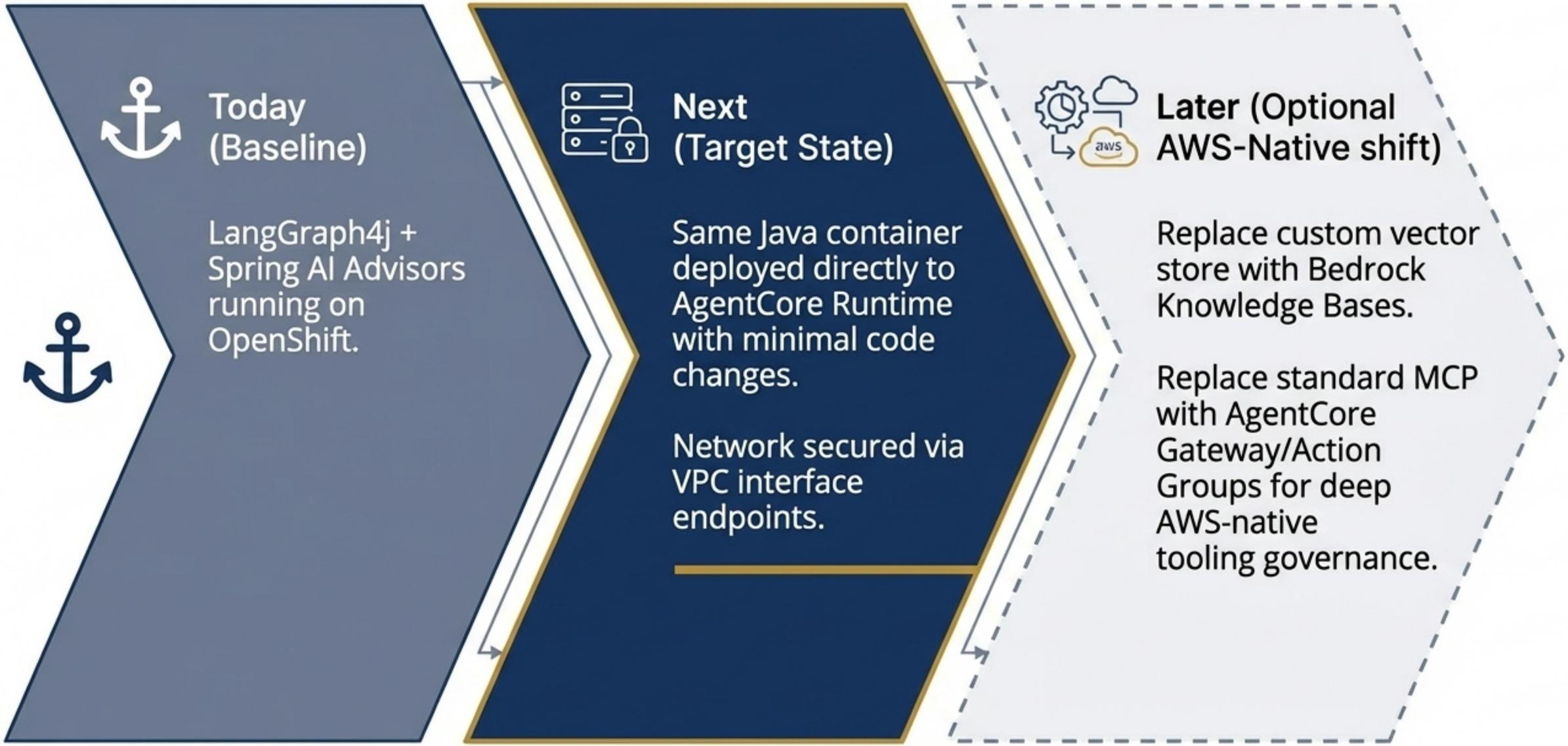
Network controls move to **VPC** + **PrivateLink** interface endpoints.

Tool access shifts to **AgentCore Gateway / MCP** servers.

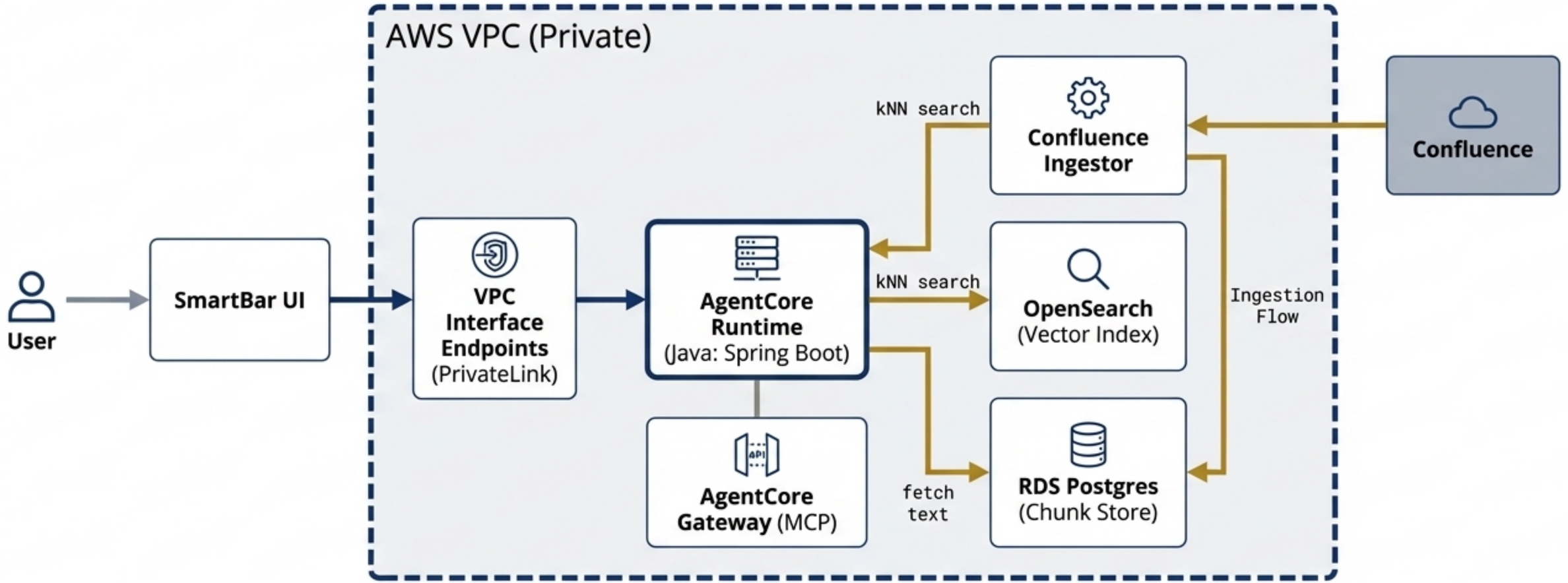
Option A Selected

Keep agent logic in Java, package as a container, and deploy to AgentCore Runtime. This is the cleanest path for existing LangGraph4j + Spring AI pipelines.

Migration Path: OpenShift to AWS-Native



Target Component Blueprint: IFEX Confluence RAG



AgentCore supports VPC connectivity and interface endpoints (PrivateLink) to ensure all calls remain entirely private and never traverse the public internet.

Non-Negotiable Bank Guardrails

ACL Filter Before LLM

Entitlements must be resolved and filtered prior to any LLM interaction. Never filter after generation.

Grounded-Only Prompting

Strict system prompts enforcing “use only provided sources.”

Citation Required

System automatically blocks answers that fail to generate valid Source Card JSON citations.

No Tool Execution in KB Query

RAG operates in strict read-only mode to prevent side-effects.

Total Auditability

Every retrieved `chunkId` and `pageVersion` must be written to central audit logs.

Development Baseline: System Requirements & Tooling

The Hardware Manifest

Compute:	8 vCPU (or Apple Siliicon equivalent).
Memory:	32 GB RAM recommended (16 GB is workable, but causes bottlenecking with local OpenSearch/Postgres + IDE overhead).
Storage:	200-300 GB free disk for Docker images, Maven repos, and logs.
Toolchain:	JDK 17+, Maven 3.9+, Docker Desktop / Podman.

Pipeline Architecture

LangGraph4j fits cleanly either as an explicit graph or an Advisor-only pipeline (`ChatClient + Advisors`).
Roboto Mono

Pro-Tip

The Single Biggest Optimization: Upgrade to 32 GB RAM. It eliminates 80% of local compilation and containerization friction.

Code to Cloud: Java Service Containerization Steps

1

Build RAG Locally (Java)

Implement strict Advisor chain order: `AclAdvisor` (SSO resolution) -> `ConfluenceRagAdvisor` (chunk fetch) -> `CitationAdvisor` (Source cards) -> `SafetyAdvisor` (grounding enforcement).

2

Containerize

Build Spring Boot jar, write Dockerfile, expose HTTP endpoint `/invoke` (matching `AgentCore`'s request/response pattern).

3

Deploy to AgentCore

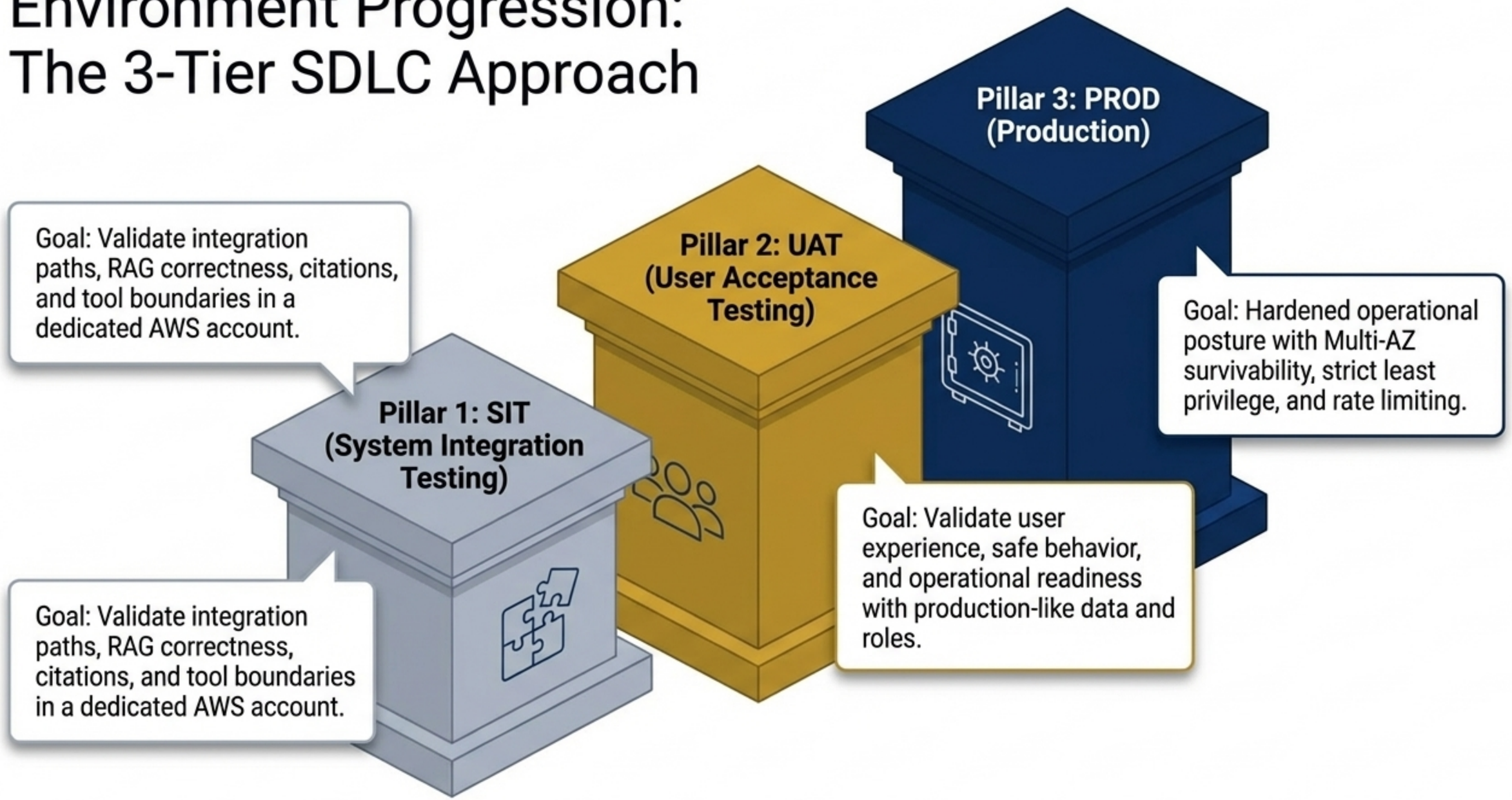
Push image to ECR -> Create AgentCore Runtime with image -> Invoke via `AgentCore` APIs using `sessionId`.

4

Connect Privately

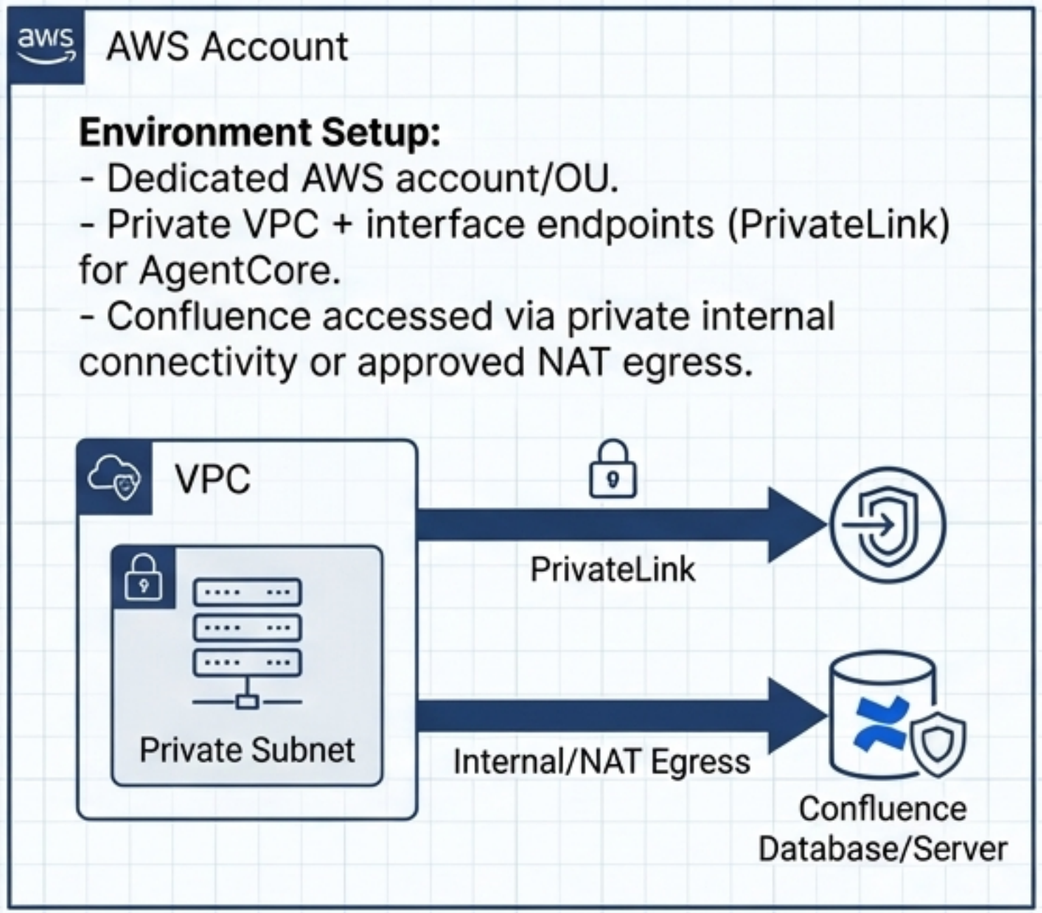
Place service inside the VPC and map AgentCore interface endpoints to guarantee private traffic routing.

Environment Progression: The 3-Tier SDLC Approach



SIT Target: System Integration & Boundary Validation

Environment Constraints



Rigorous Testing Checklist

SIT Test Focus

- [ ] Retrieval Correctness (TopK, rerank logic).
- [ ] ACL Correctness (Users only see permitted chunks).
- [ ] Citation Correctness (Links, page versions, specific sections).

Failure Mode Drills

- [ ] Vector store down.
- [ ] Confluence API throttling.
- [ ] Model unavailable (Must trigger: "Insufficient sources" fallback).

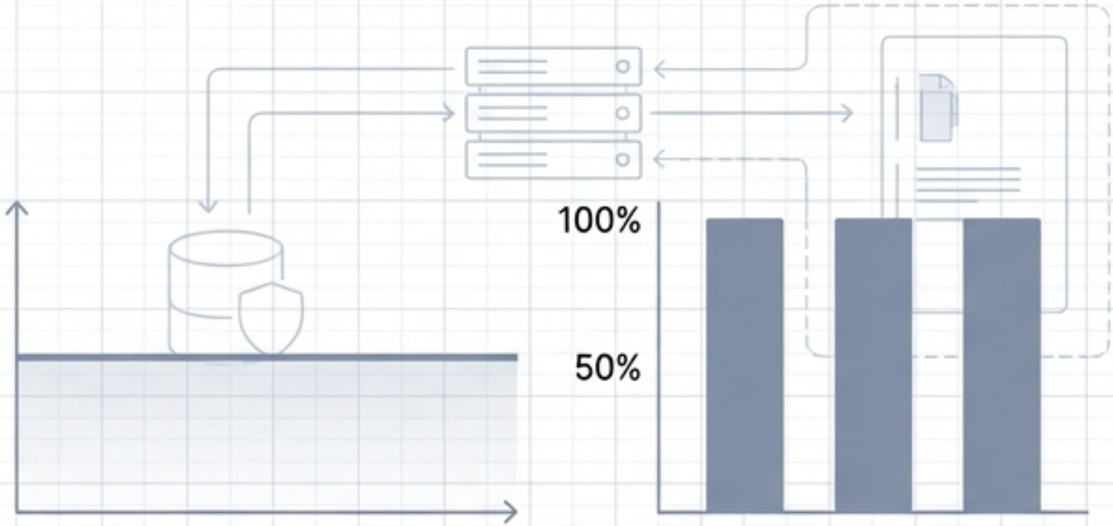
UAT Target: User Acceptance & Scale Validation

Environment Setup

- Separate AWS account.
- Sized for stable testing (`AgentCore Runtime` at 2-3 instances).
- Deployed with production-like IAM roles, audit logging, and masked/anonymized data subsets.

Observability Requirements

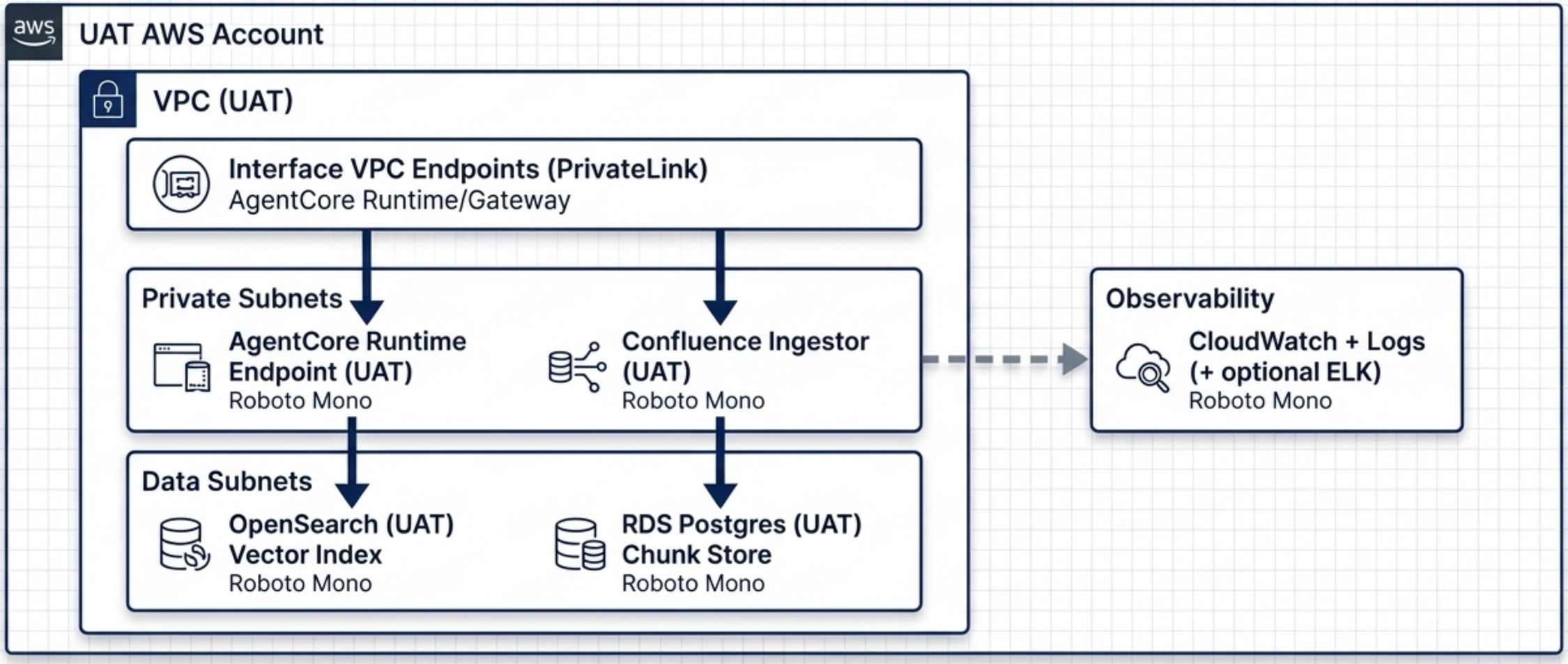
Dashboards must actively track latency, grounding rate, and cost before PROD sign-off.



Validation Focus

- **Policy Enforcement:** “Answer must cite sources” policy holds 100% of the time.
- **Hallucination Zero:** Forced abstain when not grounded.
- **Access Verification:** Role-based access validation tested against real user groups.

UAT Deployment Architecture



Note: Single region, smaller scale environment. Designed to mirror production routing but sized appropriately for QA loads and data sampling.

PROD Target: Hardened Operational Posture

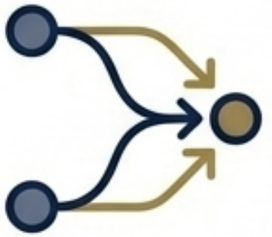
Survivability

Mandatory **Multi-AZ deployment** for both **OpenSearch** and RDS Postgres. AgentCore autoscaled based on concurrent sessions (starting at 3+ instances).

Security Perimeter

WAF and rate limits enforced at ingress. Strict **IAM least privilege** assigned to all roles.

Private subnets isolated entirely from data subnets.

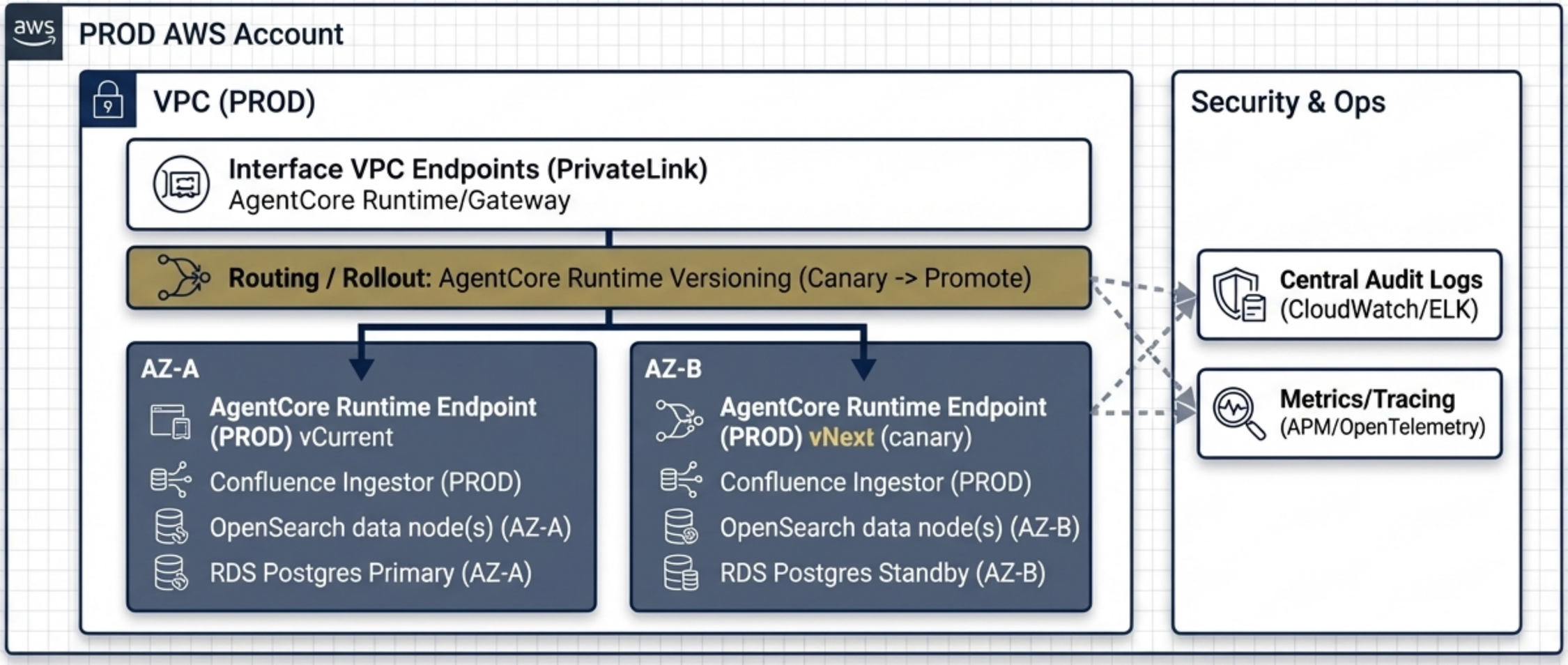


Change Management

Strict **version control** on graph configs and prompts.

`AgentCore Runtime` versioning utilized for controlled **rollouts** and **canary testing.**

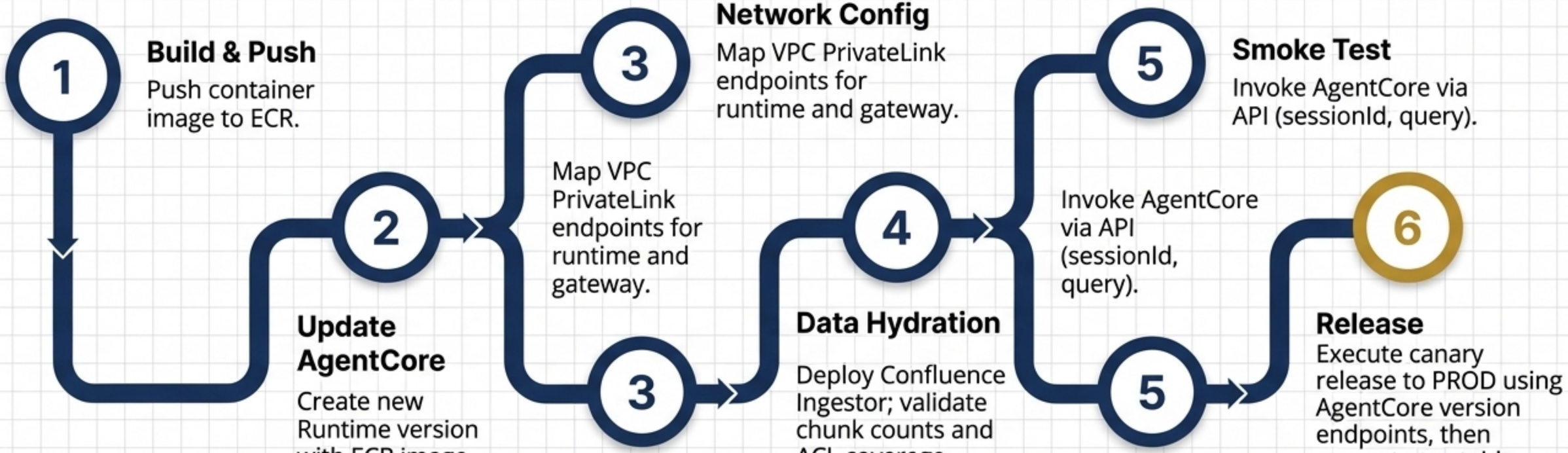
PROD Deployment Architecture (Multi-AZ & Versioning)



AgentCore Runtime supports automatic versioning. Endpoints per version allow routing traffic to canary deployments to validate performance prior to full promotion.

The AWS Deployment Sequence

Prerequisites
AWS accounts provisioned, VPC/Subnets configured, OpenSearch & RDS active, ECR ready.



Operational Runbook & Immediate Actions

Runbook Quick Reference

Health Checks	
VPCE reachability, OpenSearch/RDS p95 latency, Confluence API fetch success.	
Key Metrics	
Grounded answer rate, Abstain rate, Cost per request.	
Failure Actions	
If OpenSearch fails →	“Cannot retrieve sources” (no hallucination).
If Runtime errors →	Rollback AgentCore version.

Immediate Next Steps:

1. **Freeze UI Contract:** Lock in the answer + sources[] response JSON.
2. **Build SIT Harness:** Define golden questions, expected citations, and ACL denial sets.
3. **Deploy to SIT:** Containerize the Java RAG service and push to AgentCore Runtime.